podlodka

AI Crew
12 - 16 октября

Мультиагентные системы

Роман Алейников
ex-Teamlead в Т-Банк и EPAM

Демо-сессия

Безоблачные локальные LLM: зачем и как



RWB Selectel

На примере актуальных моделей Qwen

Qwen3.8-Flash-Next

Qwen3.8-27B

Qwen3.6-35B-A3B

ПЛАН

Четыре вещи, которые вы заберёте с собой

1 · ⚡ **Когда локальная LLM выигрывает у облака.**
Скорость, приватность, цена — и ситуации, где без неё не обойтись.

2 · 🖥 **Какое нужно железо.**
От серверных Tesla до Apple Silicon — выбор под задачу и бюджет.

3 · 🧰 **Софт и подбор модели для старта.**
Драйверы, llama.cpp и vLLM, кванты и выбор модели под VRAM.

4 · 🤖 **Мультиагентность на локальных LLM.**
Как готовить мультиагентность с помощью локальных LLM и OpenCode.

Демо: Запуск

OpenCode: один основной агент + три сабагента. Собираем веб-приложение, которое выгружает и отображает тренды репозиториев с GitHub про мульти-агентность. Доступ к GitHub — по токену из переменной среды `GITHUB_TOKEN`, которая уже задана.

Зачем нужны локальные модели

01

01 · ЗАЧЕМ

Моя мотивация

🔍 Масштабные исследования

Десятки исследований, каждое — до 100K токенов. Масштабные запросы с поиском в сети съедают бюджет подписок моментально.

🔄 Loop-engineering

Рефакторинг — часами работы, по десять циклов подряд. Лимиты облака очень быстро кончаются в таком режиме.

✅ Результат

Благодаря локальным моделям — всё удалось. Мультиагентные прогоны без лимитов токенов и прерываний.

01 · ЗАЧЕМ

Приватнее, дешевле, быстрее

Приватнее

Код, промпты и данные не покидают машину. Нет логов на стороне, нет «обучения на ваших данных» (если вы не используете сервисы аренды).

Быстрее

Выигрыш по скорости на длинной дистанции. Когда вы бы уже уперлись в лимиты подписки, локальные модели продолжат работать. Большие задачи, выполняемые в десятках циклов будут выполнены быстрее. Нет сетевых задержек и очередей провайдера. Время до первого токена и скорость зависят только от вашего железа.

Дешевле

Оборудование - разовая покупка (возможно оно у вас уже есть). Точно дешевле и окупится по сравнению с API, если вас устраивает качество генерации. По сравнению с текущими подписками может быть спорно, но только пока цены на них не выросли.

01 · ЗАЧЕМ

Ситуации, когда облако не вариант

> В каких случаях локальные LLM — не опция, а единственный вариант.

📡 Нет сети

Командировка, поездка, место без стабильного интернета — портативная AI-сборка продолжает выполнять задачи.

🔒 Конфиденциальные данные

Выход данных за периметр запрещён. Или просто не хотите, чтобы они утекали в сеть.

📈 Масштаб и объём

Длинные мультиагентные прогоны: токенный биллинг уходит в неконтролируемые цифры, лимиты растягивают задачи по времени.

🎛️ Контроль

Фиксированная версия модели, собственные кванты и конфиги — без «тихих» обновлений провайдера.

Локальные LLM выигрывают на длинной дистанции

Спринт · минуты

облако выигрывает

Короткая задача — и облако успевает первым: больше не из-за скорости генерации, а за счёт параллельных агентов. На одном агенте скорости близки.

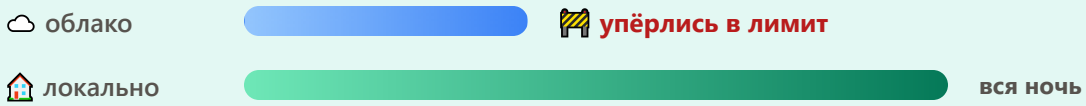


облако берёт параллельностью агентов · в рамках лимитов подписки

Марафон · часы и ночь

локальные впереди

Ночной батч на 8 часов: у облака из ограничений — 5-часовое окно, недельный лимит и пиковые часы; у локальной — только электричество.



часы агентской работы без лимитов · циклы нон-стоп

Проблемы параллельных агентов

Цепочка задач

код → тесты → деплой

Каждый шаг ждёт результата предыдущего. Параллельно — не получится.

Интеграция

1 2 3 4 5

↓

файл

Мерж, e2e-тесты, релиз — одно место на всех. Параллельные записи в один файл = конфликты до начала работы.

Невидимые конфликты

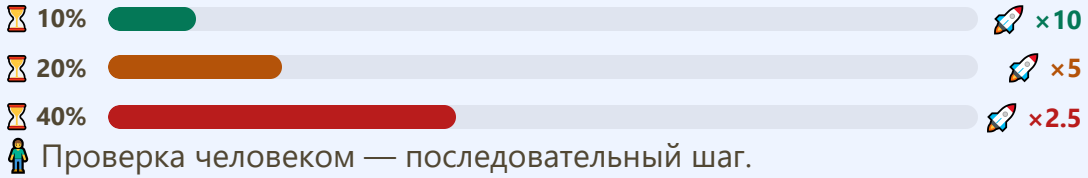
агент 1 → стиль A

агент 2 → стиль B

Параллельные агенты пишут код независимо — стили и логика могут расходиться.

Узкое место задаёт потолок ускорения

Закон Амдала: максимальное ускорение определяется не числом агентов, а долей последовательных задач.



Проверка человеком — последовательный шаг.

Параллелизм — в независимых задачах

исследования

сводки

→ один вывод

Независимые читающие задачи идут параллельно: скорость и полнота. Пишет — один агент.

Расчёты: токены / деньги / время

ВАРИАНТ	MAX ТОКЕНОВ В МЕС БЕЗ КЭША	ЦЕНА 1M ТОКЕНОВ	ТОКЕНОВ ЗА ЦЕНУ RTX 3090	МЕСЯЦЕВ ОКУПАЕМОСТИ RTX 3090
Claude API · Sonnet 5.5	-	$ 3	333 M	0,4
Claude Pro · Sonnet 5.5 · $20/мес	17,3 M	$ 1,2	833 M	1,0
Claude Max 5× · Sonnet 5.5 · $100/мес	217 M	$ 0,46	2,2 B	2,6
Claude Max 20× · Sonnet 5.5 · $200/мес	325 M	$ 0,62	1,6 B	1,9
Локально · Qwen3.8-27b (3090)	857 M	-	-	-

⚠ Расчёты оценочные. Допущения: вас устраивает качество локальной модели; вы используете локальную модель на максимум; RTX 3090 ≈ $1 000; месяц = 4,33 нед.; лимиты Claude Code — оценки (данные из сети, в т.ч. FinOps LLM); без кэша ≈ 3,9% токенов; локалка 24/7; электричество не учтено; соотношение входных и выходных токенов = 7,7:1 (200K/26K).

01 · ЗАЧЕМ

Почему ещё стоит рассмотреть локальные LLM

Подписка ≠ стабильная цена

При выжигании лимитов подписка убыточна провайдеру: то же через API стоит в разы дороже.



Закладывать подписку в unit-экономику — рискованно.
Self-hosted — себестоимость прозрачна.

Новая модель: open-weight ↑ · cloud ↓

Open-weight — новые возможности без доплаты за модель. Cloud — ценник тот же, но лимиты режут и новое заводят в платные тиры.

- **Open-weight релиз** — **↑ возможности · $0**
- **Cloud релиз** — **↓ лимиты · по токенам**

Локально всё видно



Видно реальный расход токенов на действие, нужный контекст и работу кэша.

Облаком начинаешь пользоваться **осознанно и экономно**.

Демо: Прогресс

Смотрим, что всё идёт гладко: без внештатных ситуаций, с явным прогрессом по стадиям и оценкой времени их завершения.

Подбор железа

02

02 · ЖЕЛЕЗО

Моя конфигурация

Desktop
стационарный стенд

72 GB VRAM

- **RTX 3090 Ti** 24 GB — базовая карта
- **RTX 3090** 24 GB — OCuLink · M.2
- **Tesla P40** 24 GB — OCuLink · PCI-E x4
- **i9-10900** · RAM 64 GB · SSD 2 TB

Laptop
мобильная связка

26 GB VRAM

- **RTX 3080 Ti** 16 GB — базовая карта
- **RTX 3080** 10 GB — Thunderbolt 4
- **i9-12900** · RAM 32 GB · SSD 2 TB

Задачи в зависимости от VRAM

8 GB

🧱 ТОЧКА ВХОДА

- ☕ Чат, поиск, работа с текстом
- ✍️ Простой кодинг, скрипты
- 📄 Суммаризация, перевод

Модели до 9B

12 GB

⚡ НЕБОЛЬШИЕ УМНЫЕ ЗАДАЧИ

- ⚡ Кодинг-агент в коротких сессиях
- 💬 Экспертные вопросы, вдумчивые ответы

27B · 35B-A3B

16 GB

🎯 РАБОЧАЯ ЛОШАДКА

- 🛠️ Архитектура, серьезный кодинг
- 🚀 Быстрый агент без задержек

27B и 35B-A3B — с запасом

24 GB

★ ОПТИМУМ

- 🏗️ Длинные кодинг-сессии
- 🔭 Агентные задачи, большой контекст

27B в качестве + быстрый 35B-A3B
👑 Flash-Next — работает, часть экспертов в RAM

48 GB

💎 РОСКОШНЫЙ МАКСИМУМ

- 👹 Два агента параллельно
- 🏈 Умная + быстрая модели — переключение на лету

Две модели одновременно
👑 Flash-Next — больше в VRAM, максимум качества

Не хватает VRAM — не приговор: MoE-эксперты и слои уезжают в RAM (32–64 ГБ), а модель можно запустить и на CPU + RAM (от 16 ГБ) — медленно, но работает. Для Flash-Next желательны 64 ГБ RAM и быстрый SSD.

Старые серверные NVIDIA GPU (16+GB): бюджетный VRAM

> 🗄️ Карты для дата-центров 2016–2018 — хит бюджетного LLM-инференса, если вы готовы мириться с рисками и неудобствами.

GPU	VRAM	MEMORY BANDWIDTH	MAX POWER	ARCHITECTURE	TENSOR CORES	CUDA CORES	PRICE	PRICE / 1GB
Tesla P100	**16 GB**	732 GB/s	250W	Pascal (2016)	-	3 584	**14 000 ₽**	**875 ₽**
Tesla P40	**24 GB**	346 GB/s	250W	Pascal (2016)	-	3 840	**40 000 ₽**	**1 667 ₽**
Tesla V100	**16 GB**	900 GB/s	300W	Volta (2017)	640	5 120	**22 000 ₽**	**1 375 ₽**
Tesla V100	**32 GB**	900 GB/s	300W	Volta (2017)	640	5 120	**60 000 ₽**	**1 875 ₽**
Tesla T4	**16 GB**	300 GB/s	70W	Turing (2018)	320	2 560	**60 000 ₽**	**3 750 ₽**

⚠️ Без выходов на экран и без активного охлаждения. P40/P100 требуют процессорное питание, T4 питается через PCI-слот. P100/V100 существуют как в исполнении PCIe так и в SXM2. Последние версии драйверов CUDA не поддерживают эти GPU, придется использовать более старые. Придется использовать серверные драйвера, которые могут конфликтовать с последними Game-Ready-драйверами.

Игровые NVIDIA GPU (24+GB): скорость и универсальность

> 🚀 Лучшая поддержка ПО и скорость. Последние драйвера CUDA (индустриальный стандарт). Также поддерживается Vulkan.

GPU	VRAM	MEMORY BANDWIDTH	MAX POWER	ARCHITECTURE	TENSOR CORES	CUDA CORES	PRICE	PRICE / 1GB
RTX 3090	**24 GB**	936 GB/s	350W	Ampere (2020)	328	10 496	**100 000 ₽**	**4 167 ₽**
RTX 3090 Ti	**24 GB**	1 008 GB/s	450W	Ampere (2020)	336	10 752	**120 000 ₽**	**5 000 ₽**
RTX 4090	**24 GB**	1 008 GB/s	450W	Ada (2022)	512	16 384	**200 000 ₽**	**8 333 ₽**
RTX 5090	**32 GB**	1 792 GB/s	575W	Blackwell (2025)	680	21 760	**400 000 ₽**	**12 500 ₽**

AMD GPU (24+GB): более доступная альтернатива

> 🥈 Поддержка ПО присутствует, хоть может и запаздывать по сравнению с NVIDIA.

GPU	VRAM	MEMORY BANDWIDTH	MAX POWER	ARCHITECTURE	AI ACCELERATORS	STREAM PROCESSORS	PRICE	PRICE / 1GB
RX 7900 XTX	**24 GB**	960 GB/s	355W	RDNA 3 (2022)	192	6 144	**80 000 ₽**	**3 333 ₽**
Radeon PRO W7800	**32 GB**	576 GB/s	260W	RDNA 3 (2022)	140	4 480	**140 000 ₽**	**4 375 ₽**
Radeon PRO W7900	**48 GB**	864 GB/s	295W	RDNA 3 (2022)	192	6 144	**200 000 ₽**	**4 167 ₽**
Radeon AI PRO R9700	**32 GB**	640 GB/s	300W	RDNA 4 (2025)	128	4 096	**190 000 ₽**	**5 937 ₽**

⚠ CUDA не поддерживается, только ROCm и Vulkan.

Mac mini, MacBook, Mac Studio (24+GB): портативные варианты от Apple

> 🍎 Apple Silicon с Unified Memory: нет разделения на RAM и VRAM. Портативный вариант из коробки.

GPU	VRAM	MEMORY BANDWIDTH	ARCHITECTURE	GPU CORES	PRICE	PRICE / 1GB
Mac mini M4 Pro 12-core	**24 GB**	273 GB/s	Apple Silicon	16	**160 000 ₽**	**6 667 ₽**
Mac mini M5 Pro 18-core	**64 GB**	307 GB/s	Apple Silicon	20	**420 000 ₽**	**6 563 ₽**
MacBook Pro 14" M5 Pro 15-core	**24 GB**	307 GB/s	Apple Silicon	16	**210 000 ₽**	**8 750 ₽**
MacBook Pro 14" M5 Max 18-core	**64 GB**	614 GB/s	Apple Silicon	40	**500 000 ₽**	**7 813 ₽**
Mac Studio M5 Max 18-core	**36 GB**	460 GB/s	Apple Silicon	32	**290 000 ₽**	**8 056 ₽**
Mac Studio M5 Max 30-core	**64 GB**	614 GB/s	Apple Silicon	64	**600 000 ₽**	**9 375 ₽**

⚠️ Как правило, проигрывают в скорости инференса картам RTX 3090/4090/5090. Часть памяти занимает ОС.

GPU с 24 GB VRAM из каждой категории

> ⚖️ Везде 24 GB VRAM и одна модель на всех — но скорость отличается в 4 раза. Объём памяти решает, *что* запустится, а пропускная способность и вычислительная мощность — *как быстро*.

GPU	VRAM	MEMORY BANDWIDTH	MAX POWER	ARCHITECTURE	QWEN3.8 27B TG	PRICE	PRICE / 1GB
Tesla P40	**24 GB**	346 GB/s	250W	Pascal (2016)	**10 tok/s** *	**40 000 ₽**	**1 667 ₽**
RX 7900 XTX	**24 GB**	960 GB/s	355W	RDNA 3 (2022)	**31 tok/s**	**80 000 ₽**	**3 333 ₽**
RTX 3090	**24 GB**	936 GB/s	350W	Ampere (2020)	**44 tok/s**	**100 000 ₽**	**4 167 ₽**
Mac mini M4 Pro 12-core	**24 GB**	273 GB/s	-	Apple Silicon (2020)	**22 tok/s**	**160 000 ₽**	**6 667 ₽**

⚠️ * tok/s — приблизительно: зависит от кванта, длины контекста и софта. На данном слайде без использования Speculative Decoding и MTP.

02 · ЖЕЛЕЗО

Multi-GPU: больше памяти и параллельность

> 💡 Несколько карт — это возможность как запускать модели большего размера с большим контекстом, так и запускать несколько моделей для параллельной обработки.

🧠 Больше VRAM

2× 3090/4090 → 48 GB: «роскошный максимум» — два агента, Flash-Next, длинный контекст.

🔀 Как делится

Карты делят модель — слоями и памятью контекста: влезает то, что не влезло в одну. Экспертов тоже можно разложить по картам — каждому запросу нужны лишь несколько.

🎚️ Без перегруза

Нагрузка распределяется по картам: ни одна не работает на пределе — тише и холоднее.

02 · ЖЕЛЕЗО

Multi-GPU: несколько GPU внутри одного ПК

> 💡 Когда одна карта не вытягивает — объединяем VRAM и нагрузку. Сначала честный PCIe-базис, потом NVLink.

🔌 PCIe: слоты и питание

Второй слот часто медленнее, чем выглядит, — проверь линии перед покупкой второй карты.
Каждой карте — свой кабель питания, блоку — запас.

❄️ Охлаждение

Карты греют друг друга: разноси слоты, продуй корпус, снижай вольтаж.

🖇️ NVLink: GPU-GPU канал

3090 — последние потребительские с NVLink; 4090/5090 общаются только по PCIe.

Multi-GPU: внешние видеокарты (eGPU)

> 💡 eGPU — это не только «ноутбучная» тема: VRAM и скорость добавляются снаружи и на мини-ПК, и на десктоп.

⚡ Thunderbolt / USB4: готовые доки

TB3/4 и USB4 — 40 Гбит/с, потери 10–20%; TB5 — 80 Гбит/с, потери 5–15%.
Цены РФ: DIY-корпуса TB3/4 ~15–25 тыс ₽.
Hot-plug поддерживается.

📦 OCuLink: доки

Нативный PCIe 4.0 x4 — 64 Гбит/с, потери 1–5%: почти десктопная производительность.
Цены РФ: док без БП ~9–20 тыс ₽, со встроенным БП 800 Вт ~16–25 тыс ₽.
Hot-plug нет: кабель подключать до запуска.

🧩 OCuLink: переходники

Часть со стороны компьютера: в десктопе — PCIe → OCuLink (~3–5 тыс ₽), в мини-ПК — M.2 (NVMe) → OCuLink (~2–3 тыс ₽).
Можно в слот x1, но лучше x4: у x1 пропускная способность в 4 раза меньше — видеокарта будет упираться в канал.

Демо: Прогресс

Смотрим, что всё идёт гладко: без внештатных ситуаций, с явным прогрессом по стадиям и оценкой времени их завершения.

Софтовый стек

03

Сначала фундамент: драйверы, CUDA, платформы

🎮 NVIDIA GeForce (игровые 30/40/50)

Драйвер — с nvidia.com, не через Windows Update; обычный Game Ready или Studio.
Версия драйвера определяет максимальную версию CUDA. Готовым бинарникам llama.cpp больше ничего не нужно.

NVIDIA серверные / legacy (Tesla)

Отдельная линейка драйверов — Data Center, а не привычный игровой. P40/P100/V100: остаёмся на драйвере 580 + CUDA 12.9 — CUDA 13 их уже не поддерживает. T4 и новее — поддерживаются далее.

🔴 AMD: ROCm и Vulkan

Два пути: ROCm — в основном на Linux, или сборка llama.cpp с Vulkan — работает почти на любой AMD-карте.
CUDA на AMD нет.

Apple Silicon

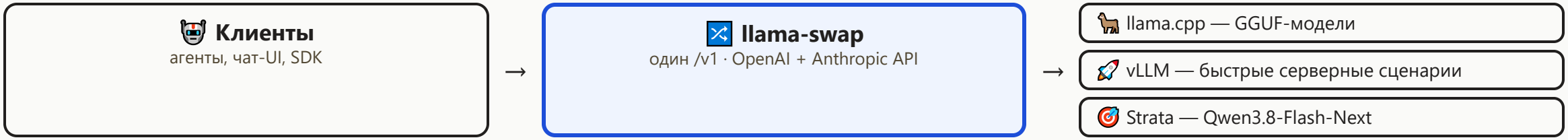
Unified Memory + Metal: драйвер ставить не надо — llama.cpp Metal и MLX из коробки (M4/M5). Часть памяти всегда держит macOS.

Движки инференса: llama.cpp, vLLM, Strata

> ⚖️ llama.cpp — домашний стенд, vLLM — серверы, Strata — Qwen3.8-Flash-Next на геймерском ПК.

	🦙 LLAMA.CPP	🚀 VLLM	🎯 STRATA
МОДЕЛИ	Лёгкие GGUF-версии: влезают в маленькую VRAM	Полноразмерные модели: максимум качества и скорости на больших GPU	Только Qwen3.8-Flash-Next (в т.ч. версии GSQ-RCO и Unsloth)
ЖЕЛЕЗО И ОС	Любой ПК: видеокарта от 8 GB или даже один CPU: квант влезает в RAM; Windows / macOS / Linux	Мощная видеокарта от 16 GB, на CPU почти не стартует; Linux, на Windows — только WSL2 или community-форки	Геймерский ПК: NVIDIA от 12 GB (RTX 20–50), RAM от 32 GB и NVMe; Windows / Linux, Mac нет
ДЛЯ ЧЕГО	Один стенд: чат и 1–5 агентов вживую	Сервис для многих: десятки и сотни запросов одновременно	Огромный MoE на одном ПК: ~20–50 tok/s там, где llama.cpp ползёт 8–20
СТАРТ	Готовые бинарники — скачал, поставил и работает	`pip install vllm` или Docker	Один клик: `setup.sh` / `START-HERE.bat` сам скачает кванты и поднимет OpenAI-сервер
ИЗ ИСХОДНИКОВ	CMake: свой бэкенд (CUDA / HIP / Vulkan / Metal) и инструкции CPU под своё железо	Возможна, но сложна: точные связки CUDA / Torch / Triton — для разработки и нестандартных платформ	C++/CUDA: ggml-ядра llama.cpp + кэш экспертов VRAM→RAM→SSD и MTP-спекуляция
ПОПУЛЯРНЫЕ ФОРКИ	ik_llama.cpp — скорость на CPU/MoE и новые кванты; в upstream они попадают с отставанием	vllm-windows — форк с нативной CUDA-сборкой на Windows (community-поддержка)	

Прокси для движков инференса: llama-swap



Хот-своп по запросу

Имя модели в запросе решает, какой бэкенд поднять. Не тот — заменит сам, без рестартов.

Один бинарь + YAML

Go без зависимостей. Установка: Docker, Homebrew, WinGet или бинари — за минуты.

Мониторинг и контроль

Веб-UI с живыми логами, метрики для Prometheus — система и GPU.

Фасады для llama.cpp: ollama, LM Studio, Unsloth

> 🎭 Движок один и тот же, фасады разные.

	🐋 OLLAMA	🎛️ LM STUDIO	🦥 UNSLOTH DESKTOP
ИНТЕРФЕЙС	CLI + API: «docker для моделей», своего GUI нет — сторонние веб-интерфейсы	GUI-приложение: чат и магазин моделей прямо в окне	GUI-приложение: чат, агенты, обучение и RAG в одном окне
МОДЕЛИ	Реестр ollama.com: `ollama pull` — модели в своём реестре на базе GGUF	Поиск и загрузка моделей с Hugging Face прямо в GUI	Свои динамические кванты + Hugging Face; Day Zero поддержка свежих релизов
ПОД КАПОТОМ	llama.cpp, на Mac ещё MLX	llama.cpp / llamafile	llama.cpp + собственный обучаемый core Unsloth
API И АГЕНТЫ	свой API + OpenAI-совместимый; SDK для Python/JS; Open WebUI и десятки интеграций	локальный сервер с OpenAI-совместимым API (LM Link); работает как MCP-клиент	OpenAI-совместимый API; `unsloth start` подключает Claude Code, Codex и других агентов; удалённый доступ через Cloudflare
ОБУЧЕНИЕ	❌ только инференс	❌ только инференс	✅ LoRA / QLoRA / RL без кода: до 2× быстрее, до 70% меньше VRAM, экспорт в GGUF
ФИШКИ	автозагрузка и версионирование моделей, минимализм	параллельные чаты, пресеты системных промптов, MCP	self-healing tool-calling, песочница для Python/Bash, web search, Deep Research, RAG, генерация картинок и видео
КОМУ	скрипты, пайплайны, автоматизация	ручной подбор, эксперименты, чат	запуск + обучение в одном окне

Демо: Прогресс

Смотрим, что всё идёт гладко: без внештатных ситуаций, с явным прогрессом по стадиям и оценкой времени их завершения.

Модели: выбор, кванты, контекст

04

Где брать модели — и три вопроса перед скачиванием

ОСНОВНОЙ ИСТОЧНИК

Hugging Face Hub

Официальные веса и карточка модели: лицензия, контекст, бенчмарки.

Форматы: **safetensors** · **GGUF** · **MLX** · **AWQ**

ГОТОВЫЕ КВАНТЫ

Фабрики квантов

На примере **Qwen3.8-27BQ4_K_M**:

unsloth — 16,2 GB
bartowski — 16,9 GB
AesSedai — 17,4 GB

ЗЕРКАЛА И ФАСАДЫ

ModelScope · Unsloth · Ollama · LM Studio

ModelScope — официальное зеркало Qwen, если HF недоступен.

Unsloth, Ollama и LM Studio качают **те же файлы** в один клик.

ТРИ ВОПРОСА ДО КНОПКИ «СКАЧАТЬ»

1 🎯 **Задача** — агент · код · чат → 2 **VRAM** — влезет ли квант → 3 **Контекст** — KV-кэш тоже ест VRAM

Архитектуры Dense и Mixture of Experts: разница в активных параметрах

🟦 Dense · Qwen3.8-27B

КАЖДЫЙ ТОКЕН · ВСЕ 27B

27B → 27B активных

Все параметры считаются на каждом токене. Топ-качество, но скорость медленнее.

🟩 MoE · Qwen3.6-35B-A3B

РОУТЕР ВЫБИРАЕТ 8 + 1 ИЗ 256

35B → ⚡ 3B активных

Весит как 35B, думает со скоростью 3B. Качество близко к крупным моделям.

🧠 Скорость задают **активные** параметры. 🚀 Агенты и интерактив — MoE · 💎 Максимум качества на сильном железе — Dense

Архитектура Qwen4 preview: Qwen3.8-Flash-Next

125B
backbone · MoE 512 экспертов/слой

+

51B
N-gram память · 2-й слой

+

4B
MTP · спекулятивный декод

=

180B
всего ·
6B
активных / токен

🔀 Attention: GDN + QSA

3 слоя **Gated DeltaNet** сжимают историю в фиксированное состояние, 1 из 4 — разрежённый attention **QSA**.

🖨️ Память: N-gram Embedding

Таблица ~20M биграмм/триграмм — **поиск по таблице**, а не вычисление; offload в RAM.

🧩 Residual: Gated Residual

Остаточный поток — **4 ветви** под поэлементным гейтом: ёмкость без лишней стоимости на токен.

⚡ MoE + MTP

10 маршрутных + 1 общий эксперт на слой; MTP ускоряет декод. Качество уровня флагмана.

Форматы весов и специальные модели

📦 Формат весов

GGUF → llama.cpp / Strata · **MLX** → Apple · **FP8 / FP16** → vLLM

Формат берут под движок.

🧪 Distilled

Знания и reasoning «дистилированы» из большой модели для маленькой — **больше качества на гигабайт**.

✂️ Abliteration

«Направление отказа» вырезают из весов **без переобучения** — инструменты `Heretic`, `DECCP`.

⚡ MTP · Multi-Token Prediction

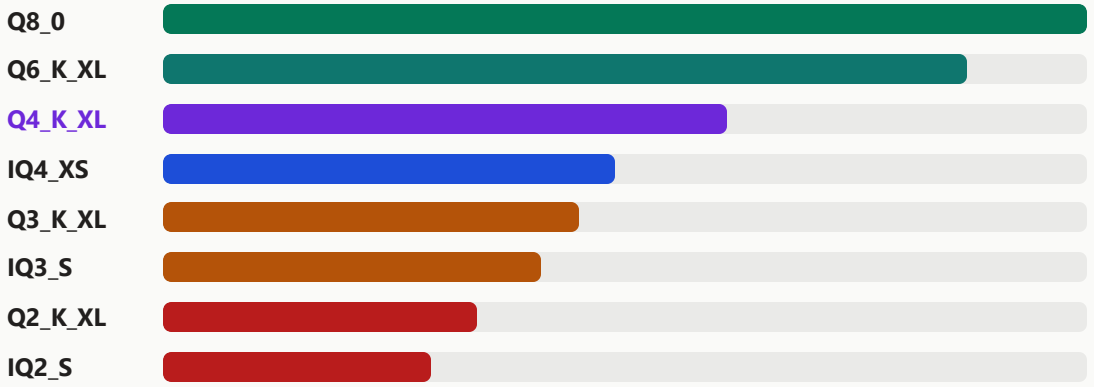
Встроенная draft-голова — speculative decoding **без отдельной драфт-модели**.

В llama.cpp включается сам по GGUF-метаданным.

Кванты: Q4 — оптимум, Q2 — лотерея

Qwen3.8-27B · Unsloth Dynamic v3.0

Размеры — реальные файлы GGUF (с MTP-модулем).



КВАНТ	ГБ	ТОЧНОСТЬ	ВЕРДИКТ
Q8_0	29	**≈100%**	эталон: на нём меряют остальные кванты
Q6_K_XL	25,3	**≈100%**	брать, если помещается в VRAM — потерь не видно
Q4_K_XL	17,6	**≈99,9%**	🎯 дефолт: почти неотличим от оригинала, втрое меньше
IQ4_XS	14,3	**≈99,8%**	тот же класс, если Q4 не влез на волосок
Q3_K_XL	13,1	**≈99%**	⚠️ компромисс: артефакты уже видны, но работать можно
IQ3_S	12	**≈98,5%**	альтернатива Q3_K_XL: минус 1 ГБ, тот же уровень
Q2_K_XL	9,8	**≈96%**	🎰 брать только с проверкой на своих задачах
IQ2_S	8,4	**≈92%**	когда больше не влезает ничего: MTP уже вырезан

📊 Точность — % сохранения качества BF16: очень приблизительная оценка, KLD-замеры Unsloth (Dynamic v3.0) переведены через парную кривую KLD→MMLU; «≈» — оценка.

Контекст определяется VRAM, ещё не потраченной на веса

💬 Чат
8K
системный промпт + последние реплики — хватает почти всегда

Кодинг-агент
32K–64K
фрагменты кода, диффы, вызовы тулз — реальный рабочий размер

Агентная сессия
128K+
память сессии, логи агентов, несколько агентов

Цена контекста: 64 KB на каждый токен для Dense-моделей

Qwen3.8-27B: полная attention только в 16 из 64 слоёв × 4 KV-головы × dim 256 × fp16; состояние 48 линейных слоёв постоянно. Растёт линейно и не «освобождается», пока токен в кэше:



Квант KV-кэша: K деликатен, V терпелив

`-ctk q8_0 -ctv q8_0` — контекст ×2 почти без потерь
`-ctk q8_0 -ctv q4_0` — ещё в полтора раза дешевле: замеры llama.cpp (KL-дивергенция) — **4-битный V почти незаметен, а 4-битный K ломает attention**
⚠ 4-битный KV-кэш заметно замедляет обработку промпта на некоторых бэкендах — экономия не бесплатна

Профили ik_llama.cpp: Qwen3.6-35b-a3b (2 GPU RTX 3090)

```
llama-server `
  --model "Qwen3.6-35B-A3B-Q8_0.gguf" `
  --gpu-layers 999 `
  --split-mode layer `
  --tensor-split 24,24 `
  --flash-attn on `
  --merge-qkv `
  --grouped-expert-routing `
  --ctx-size 32768 `
  --batch-size 2048 `
  --ubatch-size 2048 `
  --no-mmap `
  --threads 10 `
```

запуск llama-server (ik_llama.cpp)

Q8_0-файл модели 35B-A3B (MoE)

все слои — на GPU

деление модели по слоям

24/24 GB — поровну между картами

flash attention: скорость + экономия VRAM

Q/K/V склеены в один тензор (ik)

эксперты одного слоя → на один GPU

контекст 32k

виртуальный батч промпта

физический батч промпта

загрузка напрямую в RAM

CPU-потоков: 10

Профили ik_llama.cpp: Qwen3.8-27b (2 GPU RTX 3090)

```
llama-server `
  --model "Qwen3.8-27B-Q8_0.gguf" `
  --gpu-layers 999 `
  --split-mode layer `
  --tensor-split 24,24 `
  --flash-attn on `
  --merge-qkv `
  --ctx-size 32768 `
  --batch-size 2048 `
  --ubatch-size 2048 `
  --no-mmap `
  --threads 10 `
  --spec-type mtp:n_max=5,p_min=0.9 `
```

запуск llama-server (ik_llama.cpp)

Q8_0-файл модели 27B (Dense)

все слои — на GPU

деление модели по слоям

24/24 GB — поровну между картами

flash attention: скорость + экономия VRAM

Q/K/V склеены в один тензор (ik)

контекст 32k

виртуальный батч промпта

физический батч промпта

загрузка напрямую в RAM

CPU-потоки: 10

MTP speculative decoding (draft=5 токенов, confidence ≥ 0.9)

Профили ik_llama.cpp: Qwen3.8-Flash-Next (2 GPU RTX 3090) — устарел

🚧 **Этот профиль устарел.** Для Flash-Next используйте **Strata** — генерация (TG) в несколько раз быстрее — Strata написана ровно под эту модель.

```
llama-server `
  --model "Qwen3.8-Flash-Next-UD-IQ4_XS-00001-of-00003.gguf" `
  --gpu-layers 999 `
  --split-mode layer `
  --tensor-split 24,24 `
  -ot "per_layer_token_embd\.weight=CPU" `
  -ot "blk\.(0|1|2|3|4|5|6|7|8)\.ffn_(up|gate|down)_exps\.=CPU" `
  -ot "blk\.(40|41|42|43|44|45|46|47)\.ffn_(up|gate|down)_exps\.=CPU" `
  -ctk q8_0 -ctv q8_0 -ictk q8_0 `
  --flash-attn on `
  --merge-qkv `
  --grouped-expert-routing `
  --ctx-size 32768 `
  --batch-size 2048 `
  --ubatch-size 2048 `
  --no-mmap `
  --threads 10 `
```

запуск llama-server (ik_llama.cpp)

IQ4_XS shard-файл модели Flash-Next (UD, часть 1 из 3)

все слои — на GPU

деление модели по слоям

24/24 GB — поровну между картами

PLE-эмбеддинги — на CPU (экономия GPU-VRAM)

эксперты первых 9 блоков — на CPU

эксперты последних 8 блоков — на CPU

квант KV-кэша и IMD-кэша — q8_0

flash attention: скорость + экономия VRAM

Q/K/V склеены в один тензор (ik)

эксперты одного слоя → на один GPU

контекст 32k

виртуальный батч промпта

физический батч промпта

загрузка напрямую в RAM

CPU-потоков: 10

Qwen в цифрах: модель → квант → контекст → 3 показателя скорости

МОДЕЛЬ	КВАНТ	РАЗМЕР	КОНТЕКСТ	ЗАГРУЗКА	ПРОМПТ	ГЕНЕРАЦИЯ
🟩 **3.6-35B-A3B**	**Q8_0**	36.9 GB	256K	~10 с	~4000 tok/s	**~80-100 tok/s**
🟦 **3.8-27B**	**Q8_0**	29.0 GB	192K	~10 с	~1000 tok/s	**~40-60 tok/s**
👑 **3.8-Flash-Next**	GSQ-RCO **IQ3_S**	83.6 GB	128K	~50-60 с	~1500 tok/s	**~90-110 tok/s**
👑 **3.8-Flash-Next**	UD **IQ4_XS**	93.7 GB	128K	~50-60 с	~1000 tok/s	**~60-70 tok/s**

⚠️ Цифры не точные — это оценки из опыта докладчика на конфигурации 2 x RTX 3090 и 64GB RAM, проверяйте на своём железе. Вес — точный с HF: unsloth, ISTA-DASLab.

Демо: Завершение

Итоги демо: смотрим на результат и подробнее — как распределились роли агентов и кто что сгенерировал.

Локальная LLM — не эксперимент и не игрушка. Это рабочий инструмент.

При правильном подходе: быстрый, приватный, ваш. · Q&A